Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Report on laboratory work no. 3

Course: Operating Systems

Topic: The Command Line: Shell, Pipes and Text Filters

Done by:
st. gr. FAF-243, Ilie Covali

Checked by:
Reițman Patricia

Chișinău – 2026

Operating Systems · FCIM / FAF · UTM · 2026-2027

Lab 3 · The Command Line: Shell, Pipes and Text Filters

Metadata & Environment

- **Course:** Operating Systems, FCIM / FAF, UTM, Academic Year 2026-2027
- **Lab Work:** Lab 3 (Track A: The Real OS, from the Outside, Linux)
- **Author:** Ilie Covali (st. gr. FAF-243)
- **Verified by:** Reițman Patricia
- **Host OS:** Linux (CachyOS x86_64, Kernel 7.2.8-1-cachyos)
- **Shell:** GNU bash, version 5.3.3(1)-release (x86_64-pc-linux-gnu)
- **Terminal & Multiplexer:** Alacritty / Bash / tmux
- **Workspace:** `/home/ilie/Documents/SO/os-lab3` (`~/os-lab3`)

Purpose of the Work

1. Master interactive shell navigation, reverse history lookup (`Ctrl+R`), tab completion, and command documentation (`man`, `--help`, `help`).
2. Dissect the POSIX shell execution pipeline, understanding how the shell parses lines, evaluates quoting, expands braces, variables, arithmetic expressions, command substitutions, and filename wildcards (globbing) before invoking commands.
3. Direct I/O streams using redirection operators (`>`, `>>`, `<`, `2>`, `2>&1`), understand file descriptor duplication semantics, manage sequential and conditional chaining (`;`, `&&`, `||`), and control background jobs.
4. Construct composable Unix filter pipelines using `grep`, `cut`, `sort`, `uniq`, `wc`, `head`, `tail`, and `tr` to extract insights and calculate aggregations from structured access logs.
5. Author a production-grade Bash script (`report.sh`) with robust command-line argument validation, standard stream separation (`stderr` vs `stdout`), meaningful POSIX exit statuses, and automated scheduled execution via `cron`.

Setup & Dataset Initialization

1. Verification of System Environment

```
❯ whoami
ilie
❯ uname -a
Linux thinkbook 7.2.8-1-cachyos #1 SMP PREEMPT_DYNAMIC Sat, 26 Sep 2026 19:43:45 +0000 x86_64 GNU/Linux
❯ bash --version | head -1
GNU bash, version 5.3.3(1)-release (x86_64-pc-linux-gnu)
```

2. Dataset Generation (`access.log`)

The dataset represents an access log consisting of 500 requests with standardized columns: timestamp (`date time`), requesting `user`, request outcome (`OK` or `FAIL`), and accessed `page`.

```
❯ mkdir -p ~/os-lab3 && cd ~/os-lab3
❯ for i in $(seq 1 500); do
    echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $((i % 5)) -eq 0 ] && echo FAIL ||
    ↪  echo OK) /page$((i % 4))"
  done > access.log
❯ head -3 access.log ; wc -l access.log
2026-10-05 10:01 user1 OK /page1
2026-10-05 10:02 user2 OK /page2
2026-10-05 10:03 user3 OK /page3
500 access.log
```

Part 1. Move Fast, Find Help

1. Interactive Completion and History Navigation

- **Tab completion:**
  - Typing `ls /usr/sh` and pressing `Tab` immediately auto-completes the unique path prefix to `/usr/share/`.
  - Typing `cd /et` and pressing `Tab` once completes to `cd /etc/` because `/etc` is the sole directory match in the root hierarchy starting with `et`.

```
❯ ls /usr/sh<Tab>
❯ ls /usr/share/
❯ cd /et<Tab>
❯ cd /etc/
```

Figure 1.1 – Tab completion

- **Reverse Incremental History Search (`Ctrl+R`):** Pressing `Ctrl+R` invokes readline's reverse-i-search mode. Typing a substring (e.g., `access`) retrieves the most recent matching command from in-memory history:

```
(reverse-i-search)`access': head -3 access.log ; wc -l access.log
```

Pressing `Enter` executes the recalled line immediately, avoiding redundant re-typing.

2. Identifying Built-in vs. External Commands

```
❯ type cd ; type ls ; type echo ; which ls
cd is a shell builtin
ls is /usr/bin/ls
echo is a shell builtin
/usr/bin/ls
```

Figure 1.2 – Output of type and which

3. PATH Inspection & Exit Code on Missing Command

```
❯ echo $PATH | tr ':' '\n'
/home/ilie/.gemini/antigravity-cli/bin
/home/ilie/.local/bin
/home/ilie/.nvm/versions/node/v26.6.0/bin
/usr/local/bin
/usr/bin
/bin
/usr/local/sbin
/home/ilie/.local/share/flatpak/exports/bin
/var/lib/flatpak/exports/bin
/usr/lib/jvm/default/bin
/usr/bin/site_perl
/usr/bin/vendor_perl
/usr/bin/core_perl
```

Figure 1.3 – The directories in PATH

When a command cannot be located within any directory enumerated in `$PATH`:

```
❯ nosuchprog ; echo "exit code: $?"
bash: nosuchprog: command not found
exit code: 127
```

Figure 1.4 – Exit code of a missing command

4. Built-in Help and Offline Manuals

- `man ls`: Opens the man-db pager (`less`). Searching is performed by typing `/sort`, cycling forward with `n`, and terminating with `q`.
- `ls --help | head`: Displays standard command usage and option switches directly from the binary.
- `man -k "list directory"`: Queries the apropos database for keyword matches across manual section names.

- `help cd`: Outputs usage documentation for Bash built-in commands which do not have standalone man pages.

Observe (Part 1 Answers)

1. **Which of cd, ls, echo are built-ins, and why MUST cd be one?**
   - `cd` and `echo` are shell built-ins; `ls` is an external binary located at `/usr/bin/ls`.
   - `cd` **must** be a shell built-in because Unix process creation relies on `fork()` and `execve()`. If `cd` were an external program, the operating system would clone the shell into a new child process with its own independent process address space, change the current working directory of that child process, and immediately terminate upon exit. The parent shell process would retain its original working directory unchanged. Only a built-in executed within the memory context of the running shell process can modify the shell's own working directory table in the kernel (`sys_chdir`).
2. **How many directories are in your PATH, and in which one is ls?**
   - The `$PATH` environment variable contains **16** directory paths (evaluated via `echo $PATH | tr ':' '\n' | wc -l`).
   - The `ls` executable is located in `/usr/bin/ls` (with `/bin` being a symbolic link pointing to `usr/bin`).
3. **What exit code does a command that is not found return?**
   - A command not found returns exit code **127**. In POSIX standards, 127 signifies that the shell attempted to locate the executable across all entries of `$PATH` and encountered no valid binary or script.
4. **Which option of ls sorts by modification time?**
   - The `-t` option (or `--sort=time`) instructs `ls` to sort directory entries by modification timestamp, showing the newest files first.

Part 2. What the Shell Does to Your Line

1. Test Directory Setup

```
❯ mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
❯ touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
❯ ls
 a.c  'alfa beta.txt'   b.c   file1.txt   file2.txt   notes.txt
```

Figure 2.1 – Files created in the play directory

2. Prediction vs. Real Output Analysis

Before executing each command, predictions were recorded in `notes.md`. The table below outlines the comparison between predictions, real output, and the governing shell expansion mechanisms:

Command	My Prediction	Real Output	Explanation / Difference
`echo *.c`	`a.c b.c`	`a.c b.c`	Filename expansion (globbing) matches all files ending with `.c`.
`echo file?.txt`	`file1.txt file2.txt`	`file1.txt file2.txt`	Single-character wildcard `?` matches `1` and `2`.
`echo [a-b]*`	`a.c alfa beta.txt b.c`	`a.c alfa beta.txt b.c`	Character class `[a-b]` matches files starting with 'a' or 'b' (`a.c`, `alfa beta.txt`, `b.c`).

Command	My Prediction	Real Output	Explanation / Difference
`echo *.xyz`	`*.xyz`	`*.xyz`	With `nullglob` disabled, unmatched glob patterns remain literal strings.
`ls alfa beta.txt`	error: no such file	`ls: cannot access 'alfa': No such file or directoryls: cannot access 'beta.txt': No such file or directory`	Shell performs word splitting on whitespace; passes two distinct arguments.
`ls "alfa beta.txt"`	`'alfa beta.txt'`	`'alfa beta.txt'`	Double quotes suppress word splitting; single filename passed.
`echo '$HOME' "$HOME" \$HOME`	`$HOME /home/ilie $HOME`	`$HOME /home/ilie $HOME`	Single quotes prevent expansion; double quotes expand; backslash escapes `$`.
`echo {1..5} ; echo $((7 * 6))`	`1 2 3 4 542`	`1 2 3 4 542`	Brace expansion generates sequence; `$((...))` evaluates arithmetic.
`echo "today is $(date +%A), I am $(whoami)"`	`today is Monday, I am ilie`	`today is Monday, I am ilie`	Command substitution `$(...)` executes nested commands and captures stdout.
`touch log{1..3}.txt && ls log*`	`log1.txt log2.txt log3.txt`	`log1.txt log2.txt log3.txt`	Brace expansion generates three filenames created by `touch`, then listed by `ls`.

Figure 2.2 – Globbing results

```
❯ ls alfa beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
❯ ls "alfa beta.txt"
'alfa beta.txt'
❯ ls alfa\ beta.txt
'alfa beta.txt'
```

Figure 2.3 – The ls error on a name with a space, and the fix

```
❯ echo '$HOME' "$HOME" \$HOME
$HOME /home/ilie $HOME
```

Figure 2.4 – Single quotes, double quotes and backslash

Observe (Part 2 Answers)

1. **Prediction vs. Real Output:** Detailed in the table above. All predictions matched the real outputs, confirming expected POSIX shell expansion rules.
2. **echo knows nothing about files, yet echo *.c printed file names. Who produced them?**
   - The shell (Bash) produced them via pathname expansion (globbing) *before* `echo` was invoked. The shell scanned the current directory, matched filenames against `*.c`, substituted the tokens `a.c` and `b.c` on the argument vector (`argv`), and only then invoked `/bin/echo` (or the built-in) with those two parameters.
3. **Why did ls alfa beta.txt fail? Give two different ways to fix it.**
   - The shell applies word splitting based on internal field separators (`$IFS`, defaulting to space, tab, newline). Because `alfa beta.txt` contained an unquoted space, the shell split the string

into two distinct arguments: `argv[1] = "alfa"` and `argv[2] = "beta.txt"`. Neither file existed individually on disk.
- **Fix 1 (Quoting):** `ls "alfa beta.txt"` (or `ls 'alfa beta.txt'`).
- **Fix 2 (Backslash Escaping):** `ls alfa\ beta.txt`.

4. **What is the difference between single and double quotes?**
   - **Single quotes (`'...'`):** Preserve the absolute literal value of every enclosed character. Parameter expansion (`$VAR`), command substitution (`$(...)`), arithmetic expansion (`$((...))`), and backslash escapes are completely disabled.
   - **Double quotes (`"..."`):** Suppress globbing and word splitting, but preserve the special interpretation of `$`, `` ` ``, `\`, and `!` (when history expansion is active). Variables and command substitutions inside double quotes are expanded into a single argument.
5. **Why does ls show the file with a space as 'alfa beta.txt' (in quotes)?**
   - GNU `ls` defaults to `--quoting-style=shell-escape` when stdout is attached to a terminal. It surrounds filenames containing whitespace, tabs, or shell metacharacters with single quotes to signal to the user that the space is part of the filename itself and to allow safe copy-pasting directly back into the command line.

Part 3. Redirection, Chaining and Jobs

1. Directing Standard Streams

```
❯ cd ~/os-lab3
❯ ls /etc /nope > out.txt 2> err.txt
❯ wc -l out.txt ; cat err.txt
259 out.txt
ls: cannot access '/nope': No such file or directory
```

Figure 3.1 – stdout and stderr in separate files

2. Evaluating Redirection Order

We compare the evaluation order of file descriptor redirections:

```
# Order A: Redirect stdout to file, then duplicate stdout (fd 1) into stderr (fd 2)
❯ ls /etc /nope > all_a.txt 2>&1
❯ grep nope all_a.txt
ls: cannot access '/nope': No such file or directory

# Order B: Duplicate stderr from stdout (still pointing to terminal), then redirect stdout to file
❯ ls /etc /nope 2>&1 > all_b.txt
ls: cannot access '/nope': No such file or directory
❯ grep nope all_b.txt || echo "nope not in all_b.txt"
nope not in all_b.txt
```

Figure 3.2 – Comparing the order of redirections

3. Append, Stdin Redirection, and Conditional Chaining

```
❯ echo first > notes.md ; echo second >> notes.md ; cat notes.md
first
second
❯ wc -l < access.log
500
❯ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2
❯ mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
I am in /home/ilie/Documents/SO/os-lab3/tmp
```

Figure 3.3 – The ; && || operators

4. Background Job Management

```
❯ sleep 100 &
```

```
[1] 108420
❯ jobs
[1]+  Running                 sleep 100 &
❯ kill %1
[1]+  Terminated              sleep 100
❯ jobs
```

Figure 3.4 – Running a background job and stopping it

Observe (Part 3 Answers)

1. **Which file got the error message, and which got the listing?**
   - `err.txt` received the error message (`ls: cannot access '/nope': No such file or directory`) because standard error (`fd 2`) was redirected via `2> err.txt`.
   - `out.txt` received the listing of `/etc` because standard output (`fd 1`) was redirected via `> out.txt`.
2. **What is the difference between > all.txt 2>&1 and 2>&1 > all.txt? Try both and explain why the result differs.**
   - The shell processes redirection operators strictly from **left to right**.
   - In `> all.txt 2>&1`: First, file descriptor 1 (`stdout`) is redirected to point to the open file table entry for `all.txt`. Next, `2>&1` duplicates `fd 1` to `fd 2` (`stderr`), making `fd 2` also point to `all.txt`. Both stdout and stderr are written into `all.txt`.
   - In `2>&1 > all.txt`: First, `2>&1` duplicates `fd 1` to `fd 2`. At this moment, `fd 1` still points to the console terminal screen. Hence, `fd 2` is set to point to the terminal. Afterward, `> all.txt` redirects `fd 1` to `all.txt`. Consequently, stderr remains pointed to the terminal, and only stdout is captured in `all.txt`.
3. **Why does wc -l < access.log print only a number, while wc -l access.log also prints the file name?**
   - When given `access.log` as a command-line argument, `wc` opens the file by name using the `open()` system call and prints both the counted quantity and the file name argument.
   - When using `< access.log`, the shell handles the redirection, attaching `access.log` to standard input (`fd 0`). `wc` is invoked with zero filename arguments; it simply reads from its stdin stream (`fd 0`) without knowledge of the underlying file's path or name on disk.
4. **Explain each operator in one sentence: ; && || .**
   - `;` executes the subsequent command sequentially and unconditionally, regardless of the exit code of the preceding command.
   - `&&` executes the subsequent command only if the preceding command completed successfully with an exit status of 0.
   - `||` executes the subsequent command only if the preceding command failed with a non-zero exit status.

Part 4. Answer Questions with One-Liners

Every question is solved with a single pipeline using standard Unix text filters (`grep`, `cut`, `sort`, `uniq`, `wc`, `head`, `tail`, `tr`). All commands are implemented in `oneliners.sh`.

Q1. How many requests failed?

```
❯ grep -c FAIL access.log
100
```

Figure 4.1 – Number of failed requests

- **Explanation:** `grep -c FAIL` counts and prints the total number of lines matching the failure pattern. Out of 500 total lines, exactly 100 failed.

Q2. How many different pages were requested, and which?

```
❯ cut -d' ' -f5 access.log | sort -u
/page0
```

```
/page1
/page2
/page3
```

Figure 4.2 – Distinct pages requested

- **Explanation:** Field 5 contains the requested page URL. Piping into `sort -u` extracts and deduplicates the list, revealing 4 distinct endpoints (`/page0`, `/page1`, `/page2`, `/page3`).

Q3. How many requests did each page get?

```
❯ cut -d' ' -f5 access.log | sort | uniq -c
    125 /page0
    125 /page1
    125 /page2
    125 /page3
```

Figure 4.3 – Requests per page

- **Explanation:** Each page received exactly 125 requests. Verification: $125 \times 4 = 500$, perfectly matching the total line count.

Q4. Which user or users have the most failed requests, and how many?

```
❯ grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | head -2
     15 user5
     15 user3
```

Figure 4.4 – Users with most failed requests

- **Explanation:** Filtering `FAIL` records and grouping on user (field 3) demonstrates that **user3** and **user5** are tied for the highest number of failures with **15 failures each**. Full breakdown: `user0`: 14, `user1`: 14, `user2`: 14, `user3`: 15, `user4`: 14, `user5`: 15, `user6`: 14. Sum $= 14 \times 5 + 15 \times 2 = 70 + 30 = 100$ (matching Q1).

Q5. How many requests did user3 make, and how many of them failed?

```
❯ grep 'user3' access.log | cut -d' ' -f4 | sort | uniq -c
     15 FAIL
     57 OK
```

Figure 4.5 – user3 total requests and failures

- **Explanation:** `user3` initiated a total of **72 requests** ($15 + 57 = 72$), of which **15 failed** and 57 succeeded.

Q6. Print the last 3 failed requests, showing only time and user.

```
❯ grep FAIL access.log | tail -3 | cut -d' ' -f2,3
10:10 user0
10:15 user5
10:20 user3
```

Figure 4.6 – Last 3 failed requests

- **Explanation:** `grep FAIL` isolates all failures, `tail -3` selects the final three occurrences, and `cut -d' ' -f2,3` extracts only the time and username fields.

Q7. Which login shells appear in /etc/passwd, and how many accounts use each?

```
❯ cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
     38 /usr/bin/nologin
      1 /usr/bin/zsh
      1 /usr/bin/git-shell
      1 /usr/bin/bash
```

Figure 4.7 – Login shells in /etc/passwd

- **Explanation:** Field 7 of `/etc/passwd` defines the user login shell. The system contains 41 configured accounts: 38 system daemons assigned `/usr/bin/nologin`, 1 account assigned `/usr/bin/zsh`, 1 account assigned `/usr/bin/git-shell`, and 1 account assigned `/usr/bin/bash`.

Q8. PREDICT, then explain: why do ls /etc | wc -l and ls -l /etc | wc -l differ by exactly one?

```
❯ ls /etc | wc -l ; ls -l /etc | wc -l
258
259
```

Figure 4.8 – Difference between ls and ls -l

- **Prediction:** `ls -l` will return exactly 1 more line than `ls` because `ls -l` prints a directory block allocation summary header on line 1 (`total <blocks>`).
- **Explanation:** When stdout is connected to a pipe, plain `ls` emits one entry per line without headers (258 entries). In contrast, `ls -l` prints an initial metadata summary header line (e.g. `total 1300`) specifying the sum of disk blocks allocated by the directory contents, followed by the 258 entry lines. The difference is precisely that single header line ($259 - 258 = 1$).

Bonus (+1): Regex for Minute Ending in 0 and Request Failed

```
❯ grep -E ' 10:[0-9]0 .* FAIL' access.log | head -5
2026-10-05 10:10 user3 FAIL /page2
2026-10-05 10:20 user6 FAIL /page0
2026-10-05 10:30 user2 FAIL /page2
2026-10-05 10:40 user5 FAIL /page0
2026-10-05 10:50 user1 FAIL /page2
❯ grep -E ' 10:[0-9]0 .* FAIL' access.log | wc -l
50
```

- **Explanation:** Extended regular expression `' 10:[0-9]0 .* FAIL'` matches any timestamp whose minute digit ends in `0` (`10:00`, `10:10`, …, `10:50`) followed by a failure flag. Exactly **50 requests** satisfy both criteria.

Automated Run of `oneliners.sh`

```
❯ bash oneliners.sh
100
/page0
/page1
/page2
/page3
    125 /page0
    125 /page1
    125 /page2
    125 /page3
     15 user5
     15 user3
     15 FAIL
     57 OK
10:10 user0
10:15 user5
10:20 user3
     38 /usr/bin/nologin
      1 /usr/bin/zsh
      1 /usr/bin/git-shell
      1 /usr/bin/bash
258
259
2026-10-05 10:10 user3 FAIL /page2
...
```

Figure 4.9 – Full run of oneliners.sh

Part 5. Your First Script

1. Script Implementation (`report.sh`)

The completed script validates input file existence, handles missing arguments, and calculates global and per-user failure totals:

```
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
# TODO: for each of user0 ... user6, print "userN: X failures"
for i in {0..6}; do
    failures=$(grep "user$i" "$log" | grep -c FAIL)
    echo "user$i: $failures failures"
done
```

Figure 5.1 – Contents of report.sh

2. Execution and Error Handling

```
❯ chmod +x report.sh
❯ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Figure 5.2 – Running the script on access.log

When executed against a non-existent log file:

```
❯ ./report.sh nope.log ; echo "exit code: $?"
no such file: nope.log
exit code: 1
```

Figure 5.3 – Error handling for a missing file

When executed before granting execute permissions (`chmod -x`):

```
❯ chmod -x report.sh
❯ ./report.sh access.log
bash: ./report.sh: Permission denied
❯ chmod +x report.sh
```

Observe (Part 5 Answers)

1. **What does the first line #!/bin/bash do, and what happens if you run ./report.sh before chmod +x?**
   - `#!/bin/bash` is the **shebang** (hashbang) mechanism. When an executable script is invoked, the Linux kernel's `execve` binary loader inspects the first two magic bytes (`0x23 0x21`). Finding `#!`, it treats the remainder of the line as the absolute path of the interpreter program to launch, invoking `/bin/bash` and passing the script path as an argument.
   - If executed prior to running `chmod +x`, the shell attempts to issue `execve()` on the file. Because the file lacks user/group/other execute permissions (`--x`), the kernel system call fails with error `EACCES` (*Permission denied*), and Bash reports: `bash: ./report.sh: Permission denied`.
2. **Why must you type ./report.sh and not just report.sh?**

- When a command name does not contain a directory separator slash (`/`), the shell searches sequentially through the colon-delimited directories registered in the `$PATH` environment variable.
- For security reasons, the current working directory (`.` or empty entry) is intentionally excluded from `$PATH`. This prevents *trojan horse attacks*, where a malicious script named `ls` or `cd` placed in a shared directory (`/tmp`) could be inadvertently executed by another user. Typing `./report.sh` explicitly provides a relative path, bypassing PATH lookup and instructing the shell to execute the file residing in the current directory.

3. **Why does the script write the error with >&2 and end with exit 1?**
   - `>&2` redirects the error string to file descriptor 2 (standard error, `stderr`). In Unix design, diagnostic and error messages must be kept separate from computational output on `stdout` (`fd 1`). If the script's output is redirected to a file or chained into a pipeline (`./report.sh log.txt > report.out`), error messages will still appear on the user's terminal rather than corrupting the data stream.
   - `exit 1` terminates the script process with a non-zero exit status. By POSIX convention, exit code 0 denotes success, while any non-zero value ($1 \dots 255$) signals failure. This enables calling scripts or continuous integration systems to detect failures using conditional operators (`if`, `&&`, `||`).

Bonus (+1): Scheduled Execution via Cron

The script was scheduled for periodic execution by editing the user's crontab (`crontab -e`):

```
* * * * * /home/ilie/Documents/SO/os-lab3/report.sh /home/ilie/Documents/SO/os-lab3/access.log >>
↪ /home/ilie/Documents/SO/os-lab3/report.log 2>&1
```

After two minutes, the log file captures two scheduled execution cycles:

```
❯ cat ~/os-lab3/report.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Following verification, the job was removed to keep the system clean (`crontab -r`).

Conclusions

This laboratory work explored the foundational architecture of the Linux command line: 1. **Interactive Shell Navigation & Introspection:** Leveraged readline history search (`Ctrl+R`) and tab completion for rapid navigation. Clarified the architectural boundary between shell built-ins (`cd`, `echo`) that modify shell state in-process, and external utilities (`ls`) executed in separate child processes. 2. **Deterministic Shell Expansion Pipeline:** Systematically verified the order in which Bash evaluates command lines: brace expansion, tilde expansion, parameter/variable expansion, arithmetic evaluation, command substitution, word splitting, pathname expansion (globbing), and quote removal. Understood the distinct behaviors of single vs. double quotes. 3. **I/O Redirection & Process Plumbing:** Confirmed the left-to-right evaluation semantics of file descriptors (`> all.txt 2>&1` vs `2>&1 > all.txt`), mastered appending (`>>`), and orchestrated conditional and background execution (`&&`, `||`, `;`, `&`, `jobs`, `kill`). 4. **Composing Filters for Data Analysis:** Built concise, idiomatic pipelines using `grep`, `cut`, `sort`, `uniq`, and `wc` to analyze web

access logs, correctly aggregating request counts, identifying failure rates, and isolating user patterns. 5. **Robust Shell Scripting:** Implemented `report.sh` adhering to standard Unix programming practices: defensive parameter checking, stream separation (`stderr` vs `stdout`), explicit return codes, and automated execution using `cron`.